Sprint 7 · Reto «Precio y cancelación»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. El gerente de TurisData quiere **dos herramientas**. La primera es una **tarifa de referencia**: cuánto debería costar una noche según el tipo de habitación y la época (predecir un **número**: `precio_noche`). La segunda es un **aviso de riesgo de cancelación**, para decidir cuándo vender por encima de su capacidad (*overbooking*: aceptar más reservas que habitaciones, apostando a que algunas se cancelarán), que predice una **categoría**: `cancelada`, sí o no. Hay un detalle importante: **el gerente no quiere oír hablar de «AUC»**. Quiere saber **cuánto dinero se juega** si el modelo se equivoca en un sentido o en otro. Vuestro trabajo es **traducir las métricas a euros**: escribir cuánto cuesta cada tipo de error, elegir el umbral que menos cuesta, comprobar qué pasa si esos costes cambian y recomendar una acción con cifras.

Cómo se trabaja. No hace falta experiencia previa: los cuadernos del sprint (`S07_01` a `S07_04`) tienen casi todo el código que necesitáis, y vuestro trabajo es **reutilizarlo en un cuaderno nuevo** y **decidir** con él. Las hipótesis de coste son vuestras: la IA no las elige por vosotros. En clase tenéis 10 minutos de planning y **25 minutos de reto el día 1** (hipótesis de coste y precio), **30 el día 2** (cancelación, sensibilidad e informe) y **15 de entrega**; lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (Python con `pandas`, `numpy`, `scikit-learn` y `matplotlib`, las librerías del curso)	Crear y ejecutar `reto_S07.ipynb`	colab.research.google.com → *Nuevo cuaderno*
El CSV `reservas_turisdata.csv`	6000 reservas **sintéticas**; objetivos `precio_noche` y `cancelada`	Los cuadernos lo descargan solos con la función `dato()`; también en el botón *Download raw file*
Cuadernos del sprint: `S07_01` (regresión lineal) y `S07_02` (Ridge y Lasso)	Las celdas para el modelo de **precio**	Colab; su código explicado
Cuadernos del sprint: `S07_03` (logística, umbral y coste) y `S07_04` (ROC, PR y desbalance)	Las celdas para la **cancelación**; `S07_03` trae `coste_total`, el punto de partida para el coste	
Cuaderno `S07_05` (k-NN y Naive Bayes)	Solo para el segundo modelo del nivel extra	
La plantilla del informe `informe_modelado_plantilla.md`	Los apartados del informe, con huecos	
Matriz de confusión en sencillo	Repasar FN, FP y por qué los errores no cuestan igual	
Cómo entregar en GitHub	Subir la carpeta `sprint-07/` desde el navegador	
Autoevaluación y coevaluación	Cada persona rellena su hoja	
Un asistente de IA (opcional)	Entender errores y métricas	

3 · Guía paso a paso

Son cinco pasos, de **55 minutos en total** (25 el día 1 y 30 el día 2), más 15 minutos de entrega. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. Antes del paso 1 ya habéis hecho el *planning* de 10 minutos (roles del sprint y tareas en el tablero).

Paso 1 · Acordar las hipótesis de coste y abrir el cuaderno (día 1, 5 minutos)

1. Abrir un cuaderno nuevo en Colab con el nombre `reto_S07`.
2. Responder la pregunta central: ¿qué cuesta más, una cancelación no prevista o una falsa alarma?
3. Escribir una hipótesis de coste para cada error, con una frase de razón.

En sencillo. Un modelo que avisa de cancelaciones se puede equivocar de dos maneras, y **cada una cuesta dinero distinto**. Es como una aerolínea que vende más billetes que asientos: si cuenta con que algunos pasajeros no se presentan y acierta, llena el avión; si cuenta con ello y se presentan todos, tiene que compensar a quien se queda en tierra. En el hotel pasa lo mismo, y el modelo solo sirve si sabéis cuánto pesa cada error. Aquí no hay modelos todavía: abrís el cuaderno, miráis dos números de los datos (cuánto cuesta una noche y cuántas noches dura una reserva) y os ponéis de acuerdo en dos cifras en euros, cada una con su razón. No hace falta saber Python: la celda se copia y se ejecuta.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Falso negativo (FN)	El modelo dice «no se cancelará» y **sí** se cancela. La habitación queda vacía.
Falso positivo (FP)	El modelo dice «se cancelará» y **no** se cancela. Es una falsa alarma: el cliente llega y puede no haber sitio.
Hipótesis de coste	Una cifra en euros que el equipo propone para cada error, con una frase que explica de dónde sale.
Variable	Un nombre que guarda un valor, como `COSTE_FN = 284`. Se escribe en mayúsculas cuando es una cifra que fijáis vosotros.
Comentario (#)	Texto tras un #: Python lo ignora. Aquí guarda la razón de cada coste.
`DataFrame`	La tabla de datos que usa pandas. Aquí se llama `df` y tiene una fila por reserva.
`.mean()`	Calcula la media de una columna.

Cómo se hace

1. **Abrir el cuaderno.** En Colab, *Archivo* → *Nuevo cuaderno* y renombradlo `reto_S07`. Con *Compartir* (arriba a la derecha) dad acceso de editor al resto del equipo. Una persona escribe y las demás proponen. Cada bloque de código de abajo va en una **celda nueva** (*+ Código*) y se ejecuta con el botón ▶ o con Mayús + Intro, siempre de arriba abajo.

2. **Preparar y cargar los datos.** Primera celda. Es la celda «Preparación» de `S07_01`, junto con la carga del CSV:

```
import os, urllib.request
import numpy as np
import pandas as pd

BASE_URL = "https://raw.githubusercontent.com/cCoolBoox/IFCD107-alumnado/main/datos/"

def dato(nombre):
    """Devuelve la ruta local de un archivo de datos del curso (lo descarga si no está)."""
    for carpeta in ("datos", "../datos", "../../datos"):
        ruta = os.path.join(carpeta, nombre)
        if os.path.exists(ruta):
            return ruta
    os.makedirs("datos", exist_ok=True)
    ruta = os.path.join("datos", nombre)
    urllib.request.urlretrieve(BASE_URL + nombre, ruta)
    return ruta

SEMILLA = 42

df = pd.read_csv(dato("reservas_turisdata.csv"), parse_dates=["fecha_reserva", "fecha_llegada"])
df["mes_llegada"] = df["fecha_llegada"].dt.month
df["temporada_alta"] = df["mes_llegada"].isin([1, 2, 3, 7, 8, 12]).astype(int)
print(df.shape)
```

   Qué hace cada línea:

   - `import … as pd`: trae las herramientas (`pandas` para tablas, `numpy` para números). Los nombres cortos `pd` y `np` no se cambian.
   - `def dato(nombre):`: una función que busca el fichero en la carpeta del curso y, si no está, lo descarga de GitHub. No hace falta tocarla.
   - `SEMILLA = 42`: el número que fija el azar para que todo salga igual en cada ejecución.
   - `df = pd.read_csv(…)`: lee `reservas_turisdata.csv` y lo guarda en `df`. `parse_dates` avisa de que esas dos columnas son fechas.
   - `df["mes_llegada"] = …`: crea una columna nueva con el mes (1 a 12) de la fecha de llegada.
   - `df["temporada_alta"] = …`: crea otra con un 1 si el mes es enero, febrero, marzo, julio, agosto o diciembre, y un 0 si no.
   - `print(df.shape)`: escribe (filas, columnas).

3. **Entender los dos errores.** Hablad con esta tabla delante. Es la pregunta central de todo el reto:

El modelo dice	En realidad	Cómo se llama	Qué pasa en el hotel
«No se cancelará»	**Sí** se cancela	Falso negativo (FN)	La habitación queda vacía y no se vende: se pierde parte de las noches.
«Se cancelará»	**No** se cancela	Falso positivo (FP)	Se sobrevende, el cliente llega y no hay sitio: hay que compensarle.

4. **Mirar el precio medio y las noches medias.** Celda nueva. Sirve para pasar «una noche» a euros:

```
precio_medio = df["precio_noche"].mean()
noches_medias = df["noches"].mean()
print(round(precio_medio, 2), "€ por noche ·", round(noches_medias, 1), "noches por reserva")
print(round(0.5 * noches_medias * precio_medio), "€ ≈ la mitad de las noches de una reserva media")
```

   Qué hace cada línea:

   - `df["precio_noche"].mean()`: la media de la columna de precios. `noches_medias` hace lo mismo con las noches.
   - `round(x, 2)`: redondea a 2 decimales (y `round(x, 1)`, a 1). Sin el segundo número, redondea a entero.
   - `0.5 * noches_medias * precio_medio`: la mitad de lo que ingresa una reserva media. Es la cuenta del ejemplo de abajo.

5. **Escribir las hipótesis de coste.** Cada persona propone una cifra en un minuto y elegís una por cada error. Celda nueva, **con la razón al lado**:

```
COSTE_FN = 284   # cancelación no prevista: reserva media de 5,5 noches a ~103 €; solo se revende la mitad (0,5 × 5,52 × 102,87 ≈ 284 €)
COSTE_FP = 103   # falsa alarma: una noche de compensación (otro alojamiento y una disculpa) ≈ precio medio de una noche
```

   Qué hace cada línea:

   - `COSTE_FN = 284`: guarda en la variable `COSTE_FN` lo que cuesta cada falso negativo, en euros. `COSTE_FP`, lo de cada falso positivo. **Los dos nombres no se cambian**: los usan los pasos 3 y 4.
   - Lo que va tras #: la razón. Sin razón, el coste no vale.
   - Las cifras 284 y 103 son un ejemplo con estos datos. Podéis elegir otras si las razonáis, pero cada coste necesita su frase. La razón es la del enunciado: solo se revende la mitad de las noches de una reserva media, y de ahí sale la cuenta de arriba.

Qué deberíais ver. Una celda con `SEMILLA`, `COSTE_FN` y `COSTE_FP`, cada coste con su razón al lado, y el CSV cargado en un `DataFrame`. Con el CSV del curso:

- `(6000, 20)`: 6000 reservas y 20 columnas (las 18 del fichero más `mes_llegada` y `temporada_alta`).
- `102.87 € por noche · 5.5 noches por reserva`.
- `284 € ≈ la mitad de las noches de una reserva media`.

Hecho cuando: el cuaderno abre y carga los datos, y las dos hipótesis de coste están escritas con su razón.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'pd' is not defined`	La celda de preparación no se ha ejecutado (o Colab se reinició).	Ejecutadla primero y luego las demás, de arriba abajo.
`NameError: name 'df' is not defined`	Se ejecutó la celda del precio medio antes de cargar los datos.	Ejecutad la celda 2 y repetid.
`HTTPError: HTTP Error 404: Not Found`	La dirección de `BASE_URL` o el nombre del fichero tienen una errata.	Copiad de nuevo la celda y comprobad `reservas_turisdata.csv`, en minúsculas.
`KeyError: 'precio_noches'`	El nombre de la columna no es exacto.	Copiad el nombre de `df.columns`.
Una celda se ejecuta pero no muestra nada	Colab enseña solo la **última** expresión de la celda.	Envolvedlo en `print(…)`.
El equipo no se pone de acuerdo en las cifras	Cada persona imagina un hotel distinto.	Apuntad las dos hipótesis, con razón, y seguid: el paso 4 prueba qué pasa si cambian.

Paso 2 · Regresión del precio (día 1, 20 minutos)

1. Elegir las variables que **se conocen al reservar**; `cancelada` queda fuera.
2. Separar train y test, y medir el modelo de referencia (precio medio).
3. Ajustar la regresión lineal y comparar.
4. Probar una mejora y decidir si compensa. Contar el resultado en euros.

En sencillo. Vais a enseñar al ordenador a **estimar el precio de una noche** a partir de dos datos: el tipo de habitación y la época del año. Un modelo de regresión lineal es como una tarifa con suplementos: un precio base más un extra por cada característica (por ejemplo, «suite: +101 €; temporada alta: +22 €»). Para saber si vale algo, lo comparáis con un modelo de referencia que siempre dice «el precio medio». Después medís en euros cuánto se equivoca. No hace falta saber de álgebra: pandas y scikit-learn hacen las cuentas y vosotros leéis los números. Lo importante es la decisión: **qué variables se pueden usar** sin hacer trampa.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Variable objetivo	Lo que se quiere predecir. Aquí, `precio_noche`.
Variables predictoras	Las columnas con las que se predice. Aquí, el tipo de habitación y la temporada.
Fuga	Usar para predecir un dato que no se conoce al reservar, como `cancelada`. Da buenos resultados falsos.
Coeficiente	Cuánto suma o resta al precio cada característica: en una regresión lineal se lee directamente en euros.

Palabra	Qué significa, en sencillo
Variable ficticia (`get_dummies`)	Convierte una columna de texto en columnas de 0 y 1. `drop_first=True` quita una (la que queda como base).
Modelo de referencia (*baseline*)	Un modelo muy simple que sirve de suelo. Aquí, predecir siempre el precio medio de *train* (`DummyRegressor`).
Interacción	Que el efecto de una variable dependa de otra: la temporada alta puede subir más el precio de una suite que el de una estándar.
MAE y RMSE	El error en euros. MAE es el error medio; RMSE castiga más los errores grandes. Menos es mejor.
R^2	Qué parte de la variación del precio explica el modelo: 0 es nada y 1 es todo.

Cómo se hace

1. **Elegir las variables.** Para cada columna del CSV, una pregunta: «¿se conoce **al reservar**?». Escribid `df.columns.tolist()` en una celda para ver todas. La lista de usadas y excluidas va en el apartado 2 del informe. Una propuesta:

Columna	Decisión	Por qué
`precio_noche`	Es la variable objetivo	Es lo que se predice.
`cancelada`	**Fuera**	No se sabe hasta después de reservar: es fuga.
`id_reserva`	Fuera	Es un identificador; no explica el precio.
`fecha_reserva`, `fecha_llegada`	Resumidas en `temporada_alta`	Las fechas sueltas no sirven; la temporada sí.
`tipo_habitacion`, `temporada_alta`	**Dentro**	Se conocen al reservar y explican casi todo el precio.
El resto (`noches`, `canal`, `pais_origen`…)	Fuera de este modelo	Se conocen al reservar, pero apenas mejoran: con todas, el error baja de 7,24 € a unos 7,23 €.

2. **Preparar X e y y separar train y test.** Celda nueva. Los nombres llevan una p de precio para no pisar los del paso 3:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

Xp = pd.get_dummies(df[["tipo_habitacion"]], drop_first=True).astype(int)
Xp["temporada_alta"] = df["temporada_alta"]
yp = df["precio_noche"]
Xp_train, Xp_test, yp_train, yp_test = train_test_split(Xp, yp, test_size=0.2, random_state=SEMILLA)
print(Xp.shape, Xp_train.shape, Xp_test.shape)
```

   Qué hace cada línea:

   - `from … import …`: trae las piezas de scikit-learn que se usan en este paso.
   - `pd.get_dummies(df[["tipo_habitacion"]], drop_first=True)`: convierte el tipo de habitación en tres columnas de 0 y 1 (familiar, suite, superior); la estándar queda como base. `.astype(int)` asegura que son números.
   - `Xp["temporada_alta"] = …`: añade la temporada como cuarta columna.
   - `yp = df["precio_noche"]`: lo que se predice.
   - `train_test_split(…, test_size=0.2, random_state=SEMILLA)`: aparta el 20 % de las reservas como test (las que el modelo no ve al aprender). Devuelve cuatro cosas, en este orden.

3. **Medir los tres modelos.** Celda nueva. Es el máximo permitido: referencia, lineal y una mejora.

```
modelos = {
    "Referencia (media)": DummyRegressor(strategy="mean"),
    "Regresión lineal": LinearRegression(),
    "Lineal con interacciones": make_pipeline(PolynomialFeatures(degree=2, interaction_only=True, include_bias=False),
LinearRegression()),
}
filas = {}
for nombre, modelo in modelos.items():
    modelo.fit(Xp_train, yp_train)
    pred = modelo.predict(Xp_test)
    filas[nombre] = {"MAE (€)": mean_absolute_error(yp_test, pred),
                     "RMSE (€)": mean_squared_error(yp_test, pred) ** 0.5,
                     "R²": r2_score(yp_test, pred)}
resultados_precio = pd.DataFrame(filas).T.round(3)
resultados_precio
```

   Qué hace cada línea:

   - `modelos = {…}`: un diccionario con tres modelos, cada uno con un nombre. `PolynomialFeatures(…, interaction_only=True)` añade los cruces entre variables, por ejemplo suite × temporada alta.
   - `for nombre, modelo in modelos.items():`: repite lo de debajo (con la sangría) una vez por modelo. Las líneas de dentro **llevan 4 espacios**.
   - `modelo.fit(Xp_train, yp_train)`: el modelo aprende con *train*.
   - `pred = modelo.predict(Xp_test)`: predice el precio de las reservas de *test*.
   - `mean_absolute_error(yp_test, pred)`, `mean_squared_error(…) ** 0.5` y `r2_score(…)`: comparan lo predicho con el precio real y dan MAE, RMSE y R^2.
   - `pd.DataFrame(filas).T.round(3)`: junta los resultados en una tabla con una fila por modelo (`.T` la gira) y 3 decimales.

4. **Leer los coeficientes en euros.** Celda nueva:

```
lineal = modelos["Regresión lineal"]
print(pd.Series(lineal.coef_, index=Xp.columns).round(1))
print("Precio base (estándar en temporada baja):", round(lineal.intercept_, 1), "€")
```

   Qué hace cada línea:

   - `modelos["Regresión lineal"]`: recupera el modelo lineal ya ajustado.
   - `lineal.coef_`: el extra en euros de cada columna; `pd.Series(…, index=Xp.columns)` les pone nombre.
   - `lineal.intercept_`: el precio base, cuando todo vale 0 (habitación estándar y temporada baja).

5. **Decidir y escribir la frase.** Mirad la tabla: ¿la lineal gana a la referencia? ¿Compensa la mejora con interacciones? Si apenas mejora, quedaos con el modelo sencillo y decidlo en el informe. Otra mejora posible es la regularización Ridge o Lasso de `S07_02`. Escribid la frase en euros con vuestras cifras: «nos equivocamos de media en X € por noche».

Qué deberíais ver. Aproximadamente, con el CSV del curso:

- `(6000, 4) (4800, 4) (1200, 4)`: 4 columnas; 4800 reservas para entrenar y 1200 para test.
- Una tabla con una fila por modelo: **Referencia (media)** con MAE 25,564 € y R^2 -0,002; **Regresión lineal** con MAE 7,239 €, RMSE 9,704 € y R^2 0,915; **Lineal con interacciones** con MAE 6,758 €, RMSE 8,936 € y R^2 0,928.
- Coeficientes: familiar 45,0 · suite 101,2 · superior 28,4 · temporada alta 22,4, y `Precio base … 67.3 €`.
- La lineal mejora mucho a la referencia; las interacciones mejoran solo unos 0,5 € de MAE. Es lo esperado en `S07_01`: R^2 por encima de 0,85 y MAE de entre 5 y 15 €. Si el vuestro está muy lejos, mirad la «Ayuda 2» de la ficha del reto.

Hecho cuando: la tabla de precio está en el cuaderno, sin `cancelada` entre las variables, y tenéis la frase del error medio en euros.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'df' is not defined`	La celda del paso 1 no se ha ejecutado.	Ejecutad el cuaderno de arriba abajo.
`KeyError: 'tipo_habitacio'` (o el nombre que sea)	El nombre de la columna no es exacto.	Copiadlo de `df.columns`.
`ValueError: could not convert string to float: 'estandar'`	Se pasó la columna de texto sin convertirla a 0 y 1.	Usad `pd.get_dummies(…)` como en el punto 2.

Veis…	Qué significa	Qué hacer
`ValueError: Found input variables with inconsistent numbers of samples: [4800, 6000]`	X e y no tienen las mismas filas (por ejemplo, una es de *train* y la otra de todo el CSV).	Usad `Xp_train` con `yp_train`, y `Xp_test` con `yp_test`.
Las tres filas de la tabla salen casi iguales	`cancelada` o `precio_noche` se han colado en `Xp`.	`Xp.columns` debe tener solo los 3 tipos de habitación y `temporada_alta`.
Los números no coinciden con los de esta página	Otro `random_state` o otra versión de las librerías.	Mirad que `SEMILLA = 42` y esperad cifras parecidas, no idénticas.

Paso 3 · Clasificación de la cancelación y elección del umbral (día 2, 15 minutos)

1. Separar train y test, y **no tocar el test** hasta el final.
2. Medir la referencia (siempre «no cancela») y la regresión logística; dibujar la matriz de confusión y las curvas ROC y PR.
3. Elegir el umbral que minimiza el coste con validación cruzada sobre train.
4. Usar el test **una sola vez**, al final, para dar el coste esperado.

En sencillo. El modelo no dice «se cancela» o «no se cancela»: da una **probabilidad** para cada reserva (por ejemplo, 0,62). Vosotros decidís a partir de qué probabilidad hacéis saltar la alarma: ese corte es el **umbral**. Un umbral bajo avisa mucho (más falsas alarmas, menos habitaciones vacías); uno alto avisa poco. El mejor umbral no es «0,5 porque sí»: es el que **cuesta menos euros** con los costes que acordasteis en el paso 1. Se busca probando muchos umbrales con datos de entrenamiento, y el test, que es como el examen final, se abre **una sola vez** para contar cuánto ahorraríais. No hace falta entender cómo calcula la regresión logística: se usa tal cual.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Probabilidad (`predict_proba`)	Para cada reserva, un número entre 0 y 1: cuánto cree el modelo que se cancelará.
Umbral	El corte: si la probabilidad es igual o mayor, el modelo avisa de que se cancelará.
Matriz de confusión	Una tabla 2×2 que cuenta aciertos y errores: VN (no cancela y acierta), FP (falsa alarma), FN (cancelación no prevista) y VP (cancela y acierta).
Precisión y recall	Precisión: de los avisos que da el modelo, cuántos son ciertos. Recall: de las cancelaciones reales, cuántas detecta.
Coste total	FN × `COSTE_FN` + FP × `COSTE_FP`: lo que cuestan en euros los errores.
`cross_val_predict`	Da una probabilidad a cada reserva de *train* con un modelo que **no la ha visto** (se divide *train* en 5 trozos).
Curva ROC y AUC	Muestran cómo se comporta el modelo con todos los umbrales a la vez. AUC va de 0,5 (azar) a 1 (perfecto).
Curva PR y AP	Lo mismo, pero fijándose en las cancelaciones: precisión frente a recall. AP es el resumen.
Modelo de referencia (`tonto`)	`DummyClassifier(strategy="most_frequent")`: dice siempre «no cancela». Es el «no hacer nada».

Cómo se hace

1. **Separar train y test y montar los modelos.** Primera celda de este paso. Como en `S07_03`. Son 2 modelos (la referencia y la logística); el máximo es 3 por problema.

```
from sklearn.model_selection import train_test_split, cross_val_predict, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.base import clone
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, roc_curve, roc_auc_score,
                             precision_recall_curve, average_precision_score)
import matplotlib.pyplot as plt

columnas_num = ["antelacion_dias", "noches", "adultos", "ninos", "precio_noche",
                "peticiones_especiales", "cliente_repetidor", "cancelaciones_previas", "temporada_alta"]
columnas_cat = ["tipo_habitacion", "regimen", "canal", "pais_origen", "tarifa", "deposito"]
X, y = df[columnas_num + columnas_cat], df["cancelada"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEMILLA, stratify=y)

prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("esc", StandardScaler())]), columnas_num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore", drop="if_binary"))]), columnas_cat),
])
logit = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
tonto = Pipeline([("prep", clone(prep)), ("clf", DummyClassifier(strategy="most_frequent"))])
print(X_train.shape, X_test.shape, round(y_train.mean(), 3), round(y_test.mean(), 3))
```

   Qué hace cada línea:

   - `from … import …`: trae las piezas que se usan en todo el paso. `matplotlib.pyplot as plt` es para dibujar.
   - `columnas_num` y `columnas_cat`: las columnas de números y de texto que se conocen al reservar. Quedan fuera `cancelada` (es y), `id_reserva` y las fechas.
   - `X, y = …` y `train_test_split(…, stratify=y)`: apartan el 20 % para test y mantienen la misma proporción de canceladas en las dos partes.
   - `prep = ColumnTransformer([…])`: la preparación (rellenar huecos, igualar escalas, pasar texto a 0 y 1), ajustada solo con lo que le llegue a `fit`.
   - `logit` y `tonto`: la misma preparación con distinto modelo al final. `clone(prep)` hace una copia independiente.

2. **Escribir las funciones de las métricas y del coste.** Celda nueva. Ojo: aquí **no hay costes por defecto**, así que Python os obliga a pasar los vuestros (`coste_total` de `S07_03` trae FN = 3 y FP = 1).

```
def metricas_umbral(y_real, proba, umbral):
    pred = (proba >= umbral).astype(int)
    vn, fp, fn, vp = confusion_matrix(y_real, pred).ravel()
    return {"umbral": umbral, "VN": int(vn), "FP": int(fp), "FN": int(fn), "VP": int(vp),
            "precision": float(vp / (vp + fp)) if (vp + fp) else np.nan,
            "recall": float(vp / (vp + fn))}

def coste_total(y_real, proba, umbral, coste_fn, coste_fp):
    m = metricas_umbral(y_real, proba, umbral)
    return m["FN"] * coste_fn + m["FP"] * coste_fp
```

   Qué hace cada línea:

   - `pred = (proba >= umbral).astype(int)`: convierte cada probabilidad en un aviso (1) o no (0) según el umbral.
   - `confusion_matrix(y_real, pred).ravel()`: cuenta VN, FP, FN y VP, en ese orden.
   - `return {…}`: devuelve un diccionario con los cuatro conteos, la precisión y el recall.
   - `coste_total(…)`: multiplica los FN por `coste_fn` y los FP por `coste_fp`, y suma.

3. **Elegir el umbral con validación cruzada sobre train.** Celda nueva. Aquí **no se usa el test**:

```
cv = StratifiedKFold(5, shuffle=True, random_state=SEMILLA)
proba_cv = cross_val_predict(logit, X_train, y_train, cv=cv, method="predict_proba")[:, 1]
umbrales = np.arange(0.05, 0.96, 0.05)

def buscar_umbral(coste_fn, coste_fp):
    """Prueba todos los umbrales con las probabilidades de train y devuelve el de menor coste."""
    costes = []
    for u in umbrales:
        costes.append(coste_total(y_train, proba_cv, u, coste_fn, coste_fp))
    mejor = round(float(umbrales[np.argmin(costes)]), 2)
    return mejor, costes

mejor_umbral, costes = buscar_umbral(COSTE_FN, COSTE_FP)
plt.plot(umbrales, costes, marker="o")
plt.axvline(mejor_umbral, color="red", linestyle="--", label=f"Mejor umbral: {mejor_umbral}")
plt.xlabel("Umbral"); plt.ylabel("Coste total en train (€)"); plt.legend(); plt.show()
print("Umbral elegido:", mejor_umbral)
```

Qué hace cada línea:

- `cross_val_predict(…, method="predict_proba")[:, 1]`: la probabilidad de cancelar de cada reserva de *train*, calculada con un modelo que no la vio. El `[:, 1]` se queda con la columna de «cancela».
- `np.arange(0.05, 0.96, 0.05)`: los umbrales a probar, de 0,05 a 0,95 de 0,05 en 0,05.
- `def buscar_umbral(…)`: calcula el coste de cada umbral con `for` y se queda con el menor (`np.argmin`). **Se reutiliza en el paso 4.**
- `buscar_umbral(COSTE_FN, COSTE_FP)`: usa vuestros costes del paso 1. Guarda el umbral en `mejor_umbral` y la lista en `costes`.
- `plt.plot(…)` y `plt.axvline(…)`: dibujan la curva y una línea roja en el mejor umbral.

4. **Abrir el test, una vez.** Celda nueva. Con el umbral ya elegido, calcula el coste de tres formas y dibuja la matriz:

```
logit.fit(X_train, y_train)
tonto.fit(X_train, y_train)
proba_test = logit.predict_proba(X_test)[:, 1]

coste_sin_modelo = y_test.sum() * COSTE_FN
coste_05 = coste_total(y_test, proba_test, 0.5, COSTE_FN, COSTE_FP)
coste_elegido = coste_total(y_test, proba_test, mejor_umbral, COSTE_FN, COSTE_FP)
print("Coste en test sin modelo: {:,.0f} €".format(coste_sin_modelo))
print("Coste en test con umbral 0.5: {:,.0f} €".format(coste_05))
print("Coste en test con umbral {}: {:,.0f} €".format(mejor_umbral, coste_elegido))
ahorro = coste_sin_modelo - coste_elegido
print("Ahorro frente a no hacer nada: {:,.0f} € ({:.0%})".format(ahorro, ahorro / coste_sin_modelo))

m = metricas_umbral(y_test, proba_test, mejor_umbral)
print("VN {VN} · FP {FP} · FN {FN} · VP {VP}".format(**m))
print("Precisión {:.0%} · Recall {:.0%}".format(m["precision"], m["recall"]))
ConfusionMatrixDisplay.from_predictions(y_test, (proba_test >= mejor_umbral).astype(int),
                                        display_labels=["Se mantiene", "Cancela"], cmap="Blues", colorbar=False)
plt.title(f"Matriz de confusión en test (umbral {mejor_umbral})")
plt.xlabel("Lo que dice el modelo"); plt.ylabel("Lo que pasó")
plt.show()
```

Qué hace cada línea:

- `logit.fit(X_train, y_train)`: el modelo aprende con *train*. `predict_proba(X_test)` da la probabilidad de cada reserva de test.
- `coste_sin_modelo`: sin modelo nunca se avisa, así que todas las cancelaciones reales (`y_test.sum()`) son falsos negativos.
- `coste_05` y `coste_elegido`: el coste con el umbral 0,5 y con el que elegisteis.
- `"{:,.0f}".format(x)`: escribe el número sin decimales y con separador de miles (Colab usa la coma: `156,484`).
- `ConfusionMatrixDisplay.from_predictions(…)`: dibuja la matriz; el resto de líneas le ponen título y nombres.

5. **Dibujar las curvas ROC y PR** (nivel estándar; para el mínimo basta la matriz). Celda nueva, con las funciones de S07_04:

```
proba_tonto = tonto.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, proba_test)
prec, rec, _ = precision_recall_curve(y_test, proba_test)
auc, ap = roc_auc_score(y_test, proba_test), average_precision_score(y_test, proba_test)

fig, ejes = plt.subplots(1, 2, figsize=(10, 4))
ejes[0].plot(fpr, tpr, label=f"Regresión logística (AUC {auc:.2f})")
ejes[0].plot([0, 1], [0, 1], "--", color="gray", label="Azar")
ejes[0].set(title="Curva ROC", xlabel="Falsas alarmas (tasa)", ylabel="Cancelaciones detectadas (tasa)")
ejes[1].plot(rec, prec, label=f"Regresión logística (AP {ap:.2f})")
ejes[1].axhline(y_test.mean(), linestyle="--", color="gray", label="Referencia")
ejes[1].set(title="Curva PR", xlabel="Recall", ylabel="Precisión")
ejes[0].legend(); ejes[1].legend()
plt.tight_layout(); plt.show()
print(f"AUC {auc:.3f} · AP {ap:.3f} · referencia: AUC {roc_auc_score(y_test, proba_tonto):.3f}, AP
{average_precision_score(y_test, proba_tonto):.3f}")
```

   - `roc_curve(…)` y `precision_recall_curve(…)`: calculan los puntos de cada curva con todos los umbrales. El `_` guarda lo que no se usa.
   - `plt.subplots(1, 2, …)`: dos gráficos lado a lado; `ejes[0].plot(…)` dibuja en el primero y `ejes[1]` en el segundo.
   - La línea gris es la referencia: el azar en la ROC y la proporción de canceladas en la PR.

Qué deberíais ver. Con los costes del ejemplo del paso 1 (284 y 103), aproximadamente:

- `(4800, 15) (1200, 15) 0.459 0.459`: casi un 46 % de canceladas en las dos partes.
- Una curva de coste en U, con un mínimo claro, y `Umbral elegido: 0.25`.
- Coste en test: `156,484 €` sin modelo, `78,439 €` con el umbral 0.5 y `58,331 €` con el elegido, y `Ahorro frente a no hacer nada: 98,153 € (63%)`. El umbral elegido cuesta menos que el 0,5.
- `VN 204 · FP 445 · FN 44 · VP 507` y `Precisión 53% · Recall 92%`: se avisa mucho, con muchas falsas alarmas, porque un FN cuesta casi tres veces más que un FP.
- `AUC 0.746 · AP 0.729 · referencia: AUC 0.500, AP 0.459`.

Con vuestros costes saldrán otras cifras: lo que no cambia es que el mínimo es claro y que el elegido cuesta menos que el 0,5.

Hecho cuando: hay umbral elegido con vuestros costes, el coste en test está calculado una sola vez y nadie ha mirado el test para elegir el umbral.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'COSTE_FN' is not defined`	La celda del paso 1 no se ha ejecutado.	Ejecutad desde arriba.
`TypeError: coste_total() missing 2 required positional arguments: 'coste_fn' and 'coste_fp'`	Se llamó a `coste_total` sin los costes.	Pasad `COSTE_FN` y `COSTE_FP` al final de la llamada.
`ValueError: Found input variables with inconsistent numbers of samples: [1200, 4800, 1200]`	Se mezclaron probabilidades de *train* con la y de *test* (o al revés).	Con `proba_cv` va `y_train`; con `proba_test`, `y_test`.
`IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed`	Falta `method="predict_proba"` en `cross_val_predict`.	Añadidlo: sin él devuelve 0 y 1, no probabilidades.
`NotFittedError: This Pipeline instance is not fitted yet…`	Se usó `predict_proba` antes de `fit`.	Ejecutad primero `logit.fit(X_train, y_train)` (y `tonto.fit(…)`).
El umbral elegido es alto (por ejemplo 0,65) y el ahorro es pequeño	`COSTE_FN` es bajo frente a `COSTE_FP`: avisar sale caro y el modelo apenas avisa.	Revisad las razones de los costes del paso 1.

Paso 4 · Análisis de sensibilidad (día 2, 5 minutos)

1. Cambiar los costes: ¿y si una noche vacía costara el doble? ¿y si la compensación costara la mitad?
2. Calcular el umbral óptimo y el coste total de cada combinación.

3. Resumir en una tabla y una frase.

En sencillo. Los costes del paso 1 son **hipótesis**: nadie en el hotel os ha dado la cifra exacta. Antes de recomendar un umbral, conviene preguntarse: «¿y si me equivoco con el coste?». El análisis de sensibilidad es eso: repetir la búsqueda del umbral con otros costes y ver **cuánto se mueve la decisión**. Si el umbral casi no cambia, la recomendación es sólida; si cambia mucho, hay que decirlo. Es como probar una receta con más o menos sal antes de dársela a los invitados. No hay código nuevo difícil: reutilizáis la función `buscar_umbral` del paso 3 dentro de un bucle.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Análisis de sensibilidad	Repetir un cálculo cambiando una cifra de partida para ver cuánto cambia el resultado.
Bucle (`for`)	Repetir un bloque de código una vez por cada elemento de una lista.
Lista de parejas	Una lista donde cada elemento agrupa varios valores: aquí, el nombre de la hipótesis y sus dos costes.
Umbral óptimo	El umbral de menor coste para unos costes dados.
`sin_modelo`	Lo que costaría no hacer nada: todas las cancelaciones reales de *train* multiplicadas por el coste de un FN.

Cómo se hace

1. **Probar varias hipótesis de coste.** Celda nueva. Usa `buscar_umbral`, `COSTE_FN`, `COSTE_FP` y `y_train` del paso 3. Las cuatro combinaciones salen de multiplicar por 2 o por 0,5 una de las dos cifras; podéis añadir las vuestras.

```
hipotesis = [("la nuestra", COSTE_FN, COSTE_FP),
             ("FN el doble", COSTE_FN * 2, COSTE_FP),
             ("FP la mitad", COSTE_FN, COSTE_FP / 2),
             ("FP el doble", COSTE_FN, COSTE_FP * 2)]
filas = []
for nombre, coste_fn, coste_fp in hipotesis:
    umbral, costes = buscar_umbral(coste_fn, coste_fp)
    filas.append({"hipótesis": nombre, "COSTE_FN": coste_fn, "COSTE_FP": coste_fp,
                  "umbral_optimo": umbral, "coste_total": round(min(costes)),
                  "sin_modelo": round(y_train.sum() * coste_fn)})
sensibilidad = pd.DataFrame(filas)
print(sensibilidad)
```

   Qué hace cada línea:

   - `hipotesis = […]`: una lista de cuatro parejas «nombre, coste de FN, coste de FP». Los dos primeros costes de cada fila son los que se prueban.
   - `for nombre, coste_fn, coste_fp in hipotesis:`: repite lo que hay debajo (con 4 espacios) una vez por hipótesis. **Las líneas de dentro llevan sangría.**
   - `buscar_umbral(coste_fn, coste_fp)`: la función del paso 3. Devuelve el mejor umbral y la lista de costes de cada umbral.
   - `filas.append({…})`: añade una fila (un diccionario) a la lista `filas`. `min(costes)` es el coste del mejor umbral; `round(…)` lo redondea a euros.
   - `y_train.sum() * coste_fn`: el coste de no hacer nada con esa hipótesis.
   - `pd.DataFrame(filas)`: convierte la lista de filas en una tabla.

2. **Dibujar el umbral óptimo (opcional).** Un gráfico también vale. Celda nueva:

```
sensibilidad.plot.bar(x="hipótesis", y="umbral_optimo", legend=False, rot=0)
plt.ylabel("Umbral óptimo")
plt.title("Cómo se mueve el umbral si cambian los costes")
plt.show()
```

   - `sensibilidad.plot.bar(x=…, y=…)`: barras con una por hipótesis; `rot=0` deja los nombres rectos.
   - `plt.ylabel(…)`, `plt.title(…)`: nombre del eje y título. `plt.show()` dibuja.

3. **Escribir la conclusión.** Una frase con este molde y **vuestras cifras**: «Con estos costes recomendamos un umbral de X; si el coste de una falsa alarma sube al doble, el umbral sube a Y». Apuntad también en qué hipótesis cambia más la decisión.

   Una advertencia: `coste_total` de esta tabla es el de las 4800 reservas de *train*, calculado con validación cruzada. **No es comparable** con el coste en test del paso 3 (1200 reservas). Sirve para comparar hipótesis entre sí, no para dar el ahorro.

Qué deberíais ver. Una tabla de al menos tres filas (aquí cuatro) y cómo se mueve el umbral óptimo. Con los costes de ejemplo, aproximadamente:

```
     hipótesis  COSTE_FN  COSTE_FP  umbral_optimo  coste_total  sin_modelo
0   la nuestra       284     103.0           0.25       223121      625652
1  FN el doble       568     103.0           0.15       249011     1251304
2  FP la mitad       284      51.5           0.15       124506      625652
3  FP el doble       284     206.0           0.40       354720      625652
```

- Si FN se encarece, el umbral **baja** (0,25 → 0,15): se avisa más.
- Si FP se abarata, también baja (0,25 → 0,15); si se encarece, **sube** (0,25 → 0,40): se avisa menos.

Hecho cuando: la tabla (o el gráfico) está en el cuaderno y tenéis la frase de conclusión escrita.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'buscar_umbral' is not defined`	La celda del umbral del paso 3 no se ha ejecutado.	Ejecutadla antes (de arriba abajo).
`NameError: name 'COSTE_FN' is not defined`	La celda del paso 1 no se ha ejecutado.	Ejecutad desde arriba.
`IndentationError: expected an indented block after 'for' statement on line 5` (el número de línea puede ser otro)	Las líneas de dentro del `for` no tienen sangría.	Dejad 4 espacios al principio de cada línea del bucle.
`TypeError: buscar_umbral() missing 1 required positional argument: 'coste_fp'`	Se llamó a la función con un solo coste.	Pasad los dos: `buscar_umbral(coste_fn, coste_fp)`.
Todas las filas dan el mismo umbral	Se usó siempre `COSTE_FN` y `COSTE_FP` en vez de `coste_fn` y `coste_fp` del bucle.	Dentro del bucle, usad los nombres en minúscula.
La columna `coste_total` no se parece al coste en test del paso 3	Son otros datos: aquí, las 4800 reservas de *train* con validación cruzada.	No es un error; usad la tabla para comparar hipótesis, no para dar el ahorro.

Paso 5 · Informe y README (día 2, 10 minutos)

1. Copiar la plantilla como `informe_modelado.md` y rellenarla con las cifras del cuaderno.
2. Escribir el README: semilla, versiones, cómo ejecutar y uso de la IA.
3. Preparar la entrega: revisión con la rúbrica, subida y autoevaluación (los 15 minutos siguientes).

En sencillo. El trabajo técnico ya está hecho; falta **contárselo al gerente del hotel**, que no sabe qué es un AUC y quiere saber qué hacer, cuánto cuesta equivocarse y cuánto ahorra el modelo. El informe traduce vuestras cifras a euros y a una recomendación. El README es el manual para que otra persona abra el cuaderno y obtenga lo mismo. Casi todo es copiar números que ya salieron en los pasos 2 a 4 a una plantilla de texto; solo hay una celda de código nueva y es opcional. Una persona que no escribió esa parte lo revisa con la rúbrica antes de subirlo.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Informe	El documento (`informe_modelado.md`) que cuenta el trabajo a quien no programa.
README	La «portada» de la carpeta: cómo ejecutar, versiones y quién hizo qué.
Markdown (`.md`)	Texto con símbolos sencillos: ## hace un título y | separa las columnas de una tabla.
Versión	El número de edición de Python o de una librería. Con otra versión, algunas cifras pueden cambiar un poco.
Reiniciar y ejecutar todo	Borra la memoria del cuaderno y lo ejecuta de arriba abajo: prueba de que no depende de nada escondido.
Rúbrica	La tabla con la que se evalúa: cuatro criterios, cada uno de 1 a 4.
Commit	Guardar cambios en GitHub con un mensaje corto y vuestro nombre. Sin pulsar **Commit changes**, no se guarda nada.

Cómo se hace

1. **Crear y rellenar el informe.** Abrid `informe_modelado_plantilla.md`, pulsad *Raw*, copiad el texto y, en la carpeta `sprint-07` del repositorio, usad *Add file → Create new file* con el nombre `informe_modelado.md` y pegadlo. Cada hueco sale de una celda del cuaderno:

Apartado de la plantilla	Qué escribís	De dónde sale
1 · Resumen para el gerente	Seis líneas con euros: qué recomendáis, qué cuesta cada error y cuánto ahorráis frente a no hacer nada	Pasos 3 y 4
3 · Regresión	La tabla de MAE, RMSE y R^2 y la frase del error medio en euros	Paso 2 (`resultados_precio`)
4 · Clasificación	La tabla de métricas y la matriz de confusión con el umbral elegido	Paso 3
5 · De las métricas a los euros	`COSTE_FN` y `COSTE_FP` con su razón, el umbral y los tres costes en test	Pasos 1 y 3

   Esos cuatro son el **nivel mínimo**, en pocas líneas. Para el **estándar**, completad también el 2 (variables usadas y excluidas, del paso 2), la sensibilidad del 5 (la tabla del paso 4), el 6 (recomendación con cifras y límites) y el 7 (riesgos de IA responsable). Un molde para el resumen, con vuestras cifras: «Recomendamos avisar de las reservas con una probabilidad de cancelar igual o mayor que [0,25]. Con un coste de [284] € por cancelación no prevista y [103] € por falsa alarma, el modelo cuesta [58.331] € en test frente a [156.484] € sin modelo: un ahorro de [98.153] € ([63] %)». Un molde para la tabla del apartado 4:

```
| Modelo | AUC | AP | Recall (umbral) | Precisión (umbral) |
|---|---|---|---|---|
| Baseline | 0.500 | 0.459 | 0 % | — |
| Logística (umbral 0,25) | 0.746 | 0.729 | 92 % | 53 % |
```

   Pulsad *Preview* para ver cómo queda la tabla antes de guardar.

2. **IA responsable, con datos (apartado 7).** La pregunta del enunciado: ¿quién sufre los falsos positivos y se penaliza a algún canal? Para contestar con números, celda nueva (necesita `proba_test` y `mejor_umbral` del paso 3):

```
resultado = X_test.copy()
resultado["real"] = y_test
resultado["aviso"] = (proba_test >= mejor_umbral).astype(int)
no_canceladas = resultado[resultado["real"] == 0]
print(no_canceladas.groupby("canal")["aviso"].mean().round(2))
```

   Qué hace cada línea:

   - `X_test.copy()`: una copia de las reservas de test; así no se toca la original. Las dos líneas siguientes le añaden dos columnas: lo que pasó (`real`) y si el modelo avisó (`aviso`: 1 si la probabilidad llega al umbral).
   - `resultado[resultado["real"] == 0]`: se queda con las reservas que **no** se cancelaron. == significa «es igual a».
   - `.groupby("canal")["aviso"].mean()`: por cada canal, la proporción de esas reservas que recibió un aviso. Cada una es una falsa alarma. Cambiad `"canal"` por `"pais_origen"` o `"deposito"` para mirar otras.

   Con los costes de ejemplo, salen aproximadamente: `agencia 0.65`, `directo 0.49`, `empresa 0.74` y `web_intermediaria 0.85`. Es decir, un cliente que reserva por una web intermediaria y no cancela tiene mucha más probabilidad de recibir una falsa alarma que uno que reserva directo. Contadlo en el informe y proponed una medida.

3. **Escribir el README.** En `sprint-07/`, abrid `README.md` y pulsad el icono del lápiz. Cuatro cosas: `SEMILLA = 42`; las versiones; cómo ejecutar (*abrir en Colab y ejecutar de arriba abajo*); y el uso de la IA (qué pedisteis y qué revisasteis). Añadid quién hizo qué. Las versiones salen de una celda del cuaderno:

```
!python --version
import sklearn
print(sklearn.__version__)
```

   Qué hace cada línea:

   - `!python --version`: el `!` pide a Colab que ejecute esa orden del sistema y escriba la versión de Python. Solo funciona en Colab o Jupyter.
   - `import sklearn` y `print(sklearn.__version__)`: escriben la versión de scikit-learn. Copiad las dos al README; en cada ordenador salen otras y es normal.

4. **Reiniciar y ejecutar todo.** En Colab, *Entorno de ejecución → Reiniciar y ejecutar todo*, y esperad al final: ninguna celda debe mostrar un error en rojo. Si falla, corregid la primera celda en rojo y repetid.
5. **Revisión con la rúbrica (5 minutos, al empezar la entrega).** La hace una persona que **no escribió** esa parte. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica y rellena la sección «Revisión con la rúbrica» del final de `informe_modelado.md`: un nivel de 1 a 4 por criterio y **una frase de prueba** (lo que ha comprobado), más «Lo primero que mejoraríamos».

Qué deberíais ver. `informe_modelado.md` con los apartados del nivel elegido rellenos y cifras en euros, un `README.md` con semilla, versiones y cómo ejecutar, y el cuaderno entero ejecutado sin errores. La celda del punto 2 escribe cuatro líneas de canal y la del punto 3 escribe una versión de Python y otra de scikit-learn.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README, la sección «Revisión con la rúbrica» del informe está rellena y todo está subido a `sprint-07/`.

Después, 15 minutos de entrega:

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) en Colab, *Entorno de ejecución → Reiniciar y ejecutar todo* y después *Archivo → Descargar → Descargar .ipynb*; (2) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-07`**; (3) *Add file → Upload files* y arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `reto_S07.ipynb` («Añado cuaderno del reto»), la B sube `informe_modelado.md` («Añado informe»), la C edita `README.md` («Completo README»). Los nombres, **en minúsculas y sin espacios**. El CSV va a `data/` del repositorio solo si aún no está.
- **La autoevaluación.** Cada persona copia la plantilla, la rellena y la sube a `sprint-07/` como `autoevaluacion_nombre.md`. La parte de coevaluación se completa en la *review* del jueves 12/11, en el mismo fichero.
- **La review (5 minutos por equipo).** Enseñaréis la tabla comparativa de métricas, el umbral elegido y su coste en euros frente a «no hacer nada», y una frase de recomendación para el gerente.
- **La retro.** Pensad: ¿qué hipótesis de coste discutimos más? ¿Cómo cambia la decisión si cambia el coste? ¿Qué habríamos necesitado saber del hotel para afinar?

Qué deberíais ver. En la carpeta `sprint-07` del repositorio, los tres ficheros del equipo y una hoja de autoevaluación por persona (**3 + N**); y en el historial (el icono del reloj), *commits* de personas distintas.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'proba_test' is not defined`	La celda del paso 3 no se ha ejecutado antes de esta.	Ejecutad el cuaderno desde arriba con *Reiniciar y ejecutar todo*.
`SyntaxError: invalid syntax` al ejecutar la celda de las versiones	La línea `!python --version` se ejecutó fuera de Colab.	Quitad esa línea y escribid la versión de Python a mano desde `import sys; print(sys.version)`.
`KeyError: 'canal '`	El nombre de la columna lleva un espacio o una errata.	Copiadlo de `X_test.columns`.
La tabla del informe se ve rota en *Preview*	Falta una `|` en una fila o hay un salto de línea dentro de una celda.	Cada fila, en una sola línea y con las mismas `|` que la cabecera.
El archivo aparece fuera de `sprint-07/`	Se subió desde la raíz del repositorio.	Entrad primero en `sprint-07` y repetid la subida.
No sabéis qué nivel poner en la rúbrica	Los niveles están en entrega y rúbrica.	Elegid el nivel cuyo texto se cumple **entero** y escribid la prueba.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**, y también **cifras que parecen razonables y no lo son**. Reglas:

- **Las hipótesis de coste son vuestras.** Si la IA os da cifras, preguntaos de dónde salen y justificadlas. No sirve «la IA dijo 200 €».
- **No dejéis que la IA elija el umbral por vosotros:** debe salir del cálculo con vuestros costes.
- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender una métrica	«Explícame la diferencia entre MAE y RMSE con un ejemplo de precios de hotel»	Que podéis explicar cuándo conviene cada una
Entender falsos positivos y negativos	«Dame un ejemplo en un hotel de falso positivo y falso negativo de un modelo de cancelaciones»	Que los asociáis a **vuestros** costes
Depurar código	«Este código de scikit-learn da este error: *[error]*. ¿Qué lo causa?»	Que lo entendéis y lo arregláis vosotros
Revisar la recomendación	«Lee esta recomendación al gerente y dime si hay algo ambiguo o demasiado técnico»	Que no cambie las cifras

Pista de IA responsable: ¿quién sufre los falsos positivos (un cliente reubicado)? ¿Se penaliza a algún país o canal? ¿Qué pasa si el modelo se usa para pedir depósitos solo a ciertos clientes? Respondedlo en el informe.

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Usar `cancelada` o información posterior para predecir el precio	No se piensa en el momento de la reserva	Hacer una lista de variables y marcar cuáles se conocen **al reservar**
Elegir el umbral mirando el test	«Queda mejor»	El umbral sale de validación cruzada sobre train; el test, **una sola vez**
Costes sin justificar, o los de la IA	Parece un detalle	Cada coste lleva una frase de razón y es del equipo
Dejar los costes por defecto de `coste_total` (3 y 1)	Se copia la celda de `S07_03` sin cambiarla	Pasar `COSTE_FN` y `COSTE_FP` a la función
Informe solo con AUC	Es lo que muestra el cuaderno	Traducir a euros: «el modelo ahorra X € frente a no hacer nada»
Probar 10 modelos	«Más es mejor»	Máximo **3 modelos por problema**, referencia incluida
Olvidar el análisis de sensibilidad	Parece opcional	Es lo que da robustez a la recomendación
El cuaderno da errores de dimensiones	La variable objetivo sigue dentro de `X`, o `X` e `y` no coinciden	Comprobar `X.shape` y `y.shape` y que `precio_noche` o `cancelada` no están en `X`

Ayuda 3 · Si os atascáis

1. **No sabemos qué costes poner:** pensad en una noche de un alojamiento de TurisData, razonad con una frase y luego **variad** los costes en la sensibilidad.
2. **No entendemos el umbral:** pensad en un semáforo. Con el umbral muy bajo avisamos casi siempre (muchas falsas alarmas); muy alto, casi nunca (muchas cancelaciones sin detectar).
3. Repasad el cuaderno de esa parte (`S07_01` a `S07_04`) y su código explicado. Después preguntad al equipo y, por último, al docente.